wg-client Documentation

Release 8.0.3

Gene C

Sep 03, 2026

WIREGUARD CLIENT TOOL MANUAL:

CHAPTER

ONE

WG-CLIENT

Migration required from pre 8.x versions

Here's what is needed:

- Create /etc/wg-client/wireguard-resolv.conf This should be the DNS settings to be used while wireguard is running. Standard resolv.conf format (nameserver x.x.x.x)
- confirm that /etc/wg-client/config exists and sets thw wireguard i interface. For example:

```
iface = wg0
```

- edit the wireguard client config and update the PostUp and PostDn. For example:

```
[Interface]
PrivateKey = ...
Address = ...
Postup = /etc/wg-client/post-up.sh
PostDown = /etc/wg-client/post-down.sh
```

Thats should be all that's required.

- There is a new *resolv-manager*

1.1 Overview

wg-client makes it simple for any user to start and stop a wireguard client on Linux. The command line program, *wg-client*, provides all the functionality. A GUI tool makes it super convenient. The GUI tool runs the command line client and thus offeres exactly the same functionality in a user friendly application.

A companion package, wg-tool is for adminstering the server side of things.

wg-client also has an option to invoke ssh to create a remote listening port connected back to local (client end) ssh daemon.

This can be useful to facilitate remote ssh back to client computer if it's needed. For example; it can be used to provide access to a git repo on the client, or for remote backups of laptop, or even for admin to login to client should the need arise.

All git tags are signed with arch@sapience.com key which is available via WKD or download from https://www.sapience.com/tech. Add the key to your package builder gpg keyring. The key is included in the Arch package and the source= line with *?signed* at the end can be used to verify the git tag. You can also manually verify the signature

1.2 Why I made wg-client

After building wg-tool which simplified administraion of wireguard servers, I needed a simple way for non-tech (unprivileged) users to connect their laptops to the server.

Thus wg-client was born. The gui client makes it really simple for non-tech users, though I find it pretty convenient as well.

1.3 Key features

- Graphical app makes it simple for any user to get VPN running.
- Standalone tool makes it easy to test and also keeps sudo outside of gui to minimize any security implications. The gui relies completely on the command line tool to do the real work.
- resolv-manager is written on C. It uses capabilities and is designed to be run by non-root users. When run as root, it drops privileges to (user, group) = (*nobody*, *nobody*).. Code has been checked and is clean using: - clang-tidy - valgrind - compiler based sanitizers
- python code is checked with mypy

CHAPTER

TWO

GETTING STARTED

2.1 DNS Notes

When using a any VPN client, it is standard practive to make sure that all DNS traffic is routed through the tunnel in order to trusted DNS servers. While this is not a hard requirement, and may not always be applicable with split routing, it is always desirable from the security standpoint.

There is a little gotcha to be aware of. The system may, at times, modify */etc/resolv.conf* removing the safety of the VPN provided DNS servers by replacing this file.

Consequently it is important to be aware if this occurrs. The way it is handled is by saving this just updated file and restore the one that should be used with wireguard. The one that just got saved is the the right one to use when the VPN is turned off and by saving a copy of it, it can be restored to /etc/resolv.conf when wireguard shuts down.

What causes this to happen? It can happen for different reasons. For example if WiFi network changes, or after a sleep resume cycle or when a dhcp lease is renewed.

While wireguard itself uses UDP and is largely indifferent to such network, it is important to keep DNS properly managed.

Taking care of all of this is eactly what *resolv-manager* does.

To ensure that DNS resolver files are all managed correctly requires a copy of the standard resolv.conf file saved to */etc/resolv.conf.saved* and the wireguard version to */etc/resolv.conf.wg*.

With those in place *resolv-manager* will keep track of any changes. It will saev any new standard */etc/rsolv.conf* and put back the appropriate one for wireguard.

After creating /etc/wg-client/wireguard-resolv.conf which has the nameservers using the DNS servers provided by the wireguard server end, then set wireguard's configuration file (*/etc/wireguard/wgc.conf* for example) to use PostUp and PosDown. Check that the wg-client config file, /etc/wg-client/config, sets the wireguard interface. For example with a line:

```
iface = wg0
```

```
[Interface]
PrivateKey = ...
Address    = ...
Postup     = /etc/wg-client/post-up.sh
PostDown   = /etc/wg-client/post-down.sh
...
```

There helper scripts *post-up.sh* and *post-down.sh* are part of wg-client. wg-client handles everything else including running resolv-manager to monitor /etc/resolv.conf while thw vpn is active to ensure the resolv.conf file remains correct.

More information about resolv-manager can be found in the man page:

```
man wg-client-resolv-manageer
```

2.2 wg-client application

2.2.1 Usage

wg-client is a command line program and can be run in any terminal. It has two primary options, one to start wireguard and one to stop it.

```
wg-client --wg-up
wg-client --dg-dn
```

It also has a config file (/etc/wg-client/config) where the wireguard interface is specified and ssh information if that is being used.

To get a list of all options use *-h*.

The command line options for *wg-client* are described in the manual section *Options*. The configuration file contents in section *Configuration*.

2.3 wg-client-gui application

2.3.1 GUI Usage

The gui app can be run using the wg-client.desktop file and can be added to launchers in the usual way. For example in gnome simply search applications for wg-cliient and right click to pin the launcher. The gui is built on PyQt6 which in relies on Qt6.

The gui has buttons to start and stop wireguard and a button to run ssh to set up the listener on the host configured in the config file.

The gui should be left running while the vpn is in use. Pressing quit in the gui will shutdown wireguard and shutdown the ssh listener as well.

2.4 Sudoers

To start and stop wireguard wg-client uses *wg-quick* (in wireguard-tools). Doing so requires root privieles and so any user approved to run wireguard must be granted permission. Any non-root user will need a NOPASSWD sudoers entry.

You can keep all local sudoers in a single file or in separate files. If in single file, make this one come after any group wheel ones. This is to ensure this one is chosen becuase sudo uses the last matching entry.

Simply add this sample line adjusting WGUSERS to list whatever user(s) are permitted to run wireguard. If more than one use comma separated list as shown below.

```
User_Alias WGUSERS = alice, bob, sally
WGUSERS   ALL = (root) NOPASSWD: /usr/bin/wg-quick
WGUSERS   ALL = (root) NOPASSWD: /usr/lib/wg-client/wg-fix-dns
```

If using separete files, then care is need to ensure this entry comes after any wheel group entries. Where WGUSERS is 1 or more usernames or a group such as *%wgusers*.

Then,

```
visudu /etc/sudoers.d/100-wireguard
```

Edit *WGUSERS* as above.

visudo enforces the correct permissions which should be '0440'. If permissions are too loose, sudo will ignore the file.

Why the prefix number? Because sudo uses the **last** matching entry and we need to be sure the NOPASSWD wg-quick entry comes after any group wheel lines.

For example if there are 2 files in */etc/sudoers.d* - say wg-quick and wheel, where the wheel entry requires a password for members of group wheel.

Now if user listed in wg-quick is also a member of *wheel* group, since wg-quick is first and wheel is second (files are treated in lexical order) the *wheel* one will prevail and user will be prompted for a password when running *sudo /usr/bin/wg-quick*. Not what we want. To fix this use numbers to prefix the sudoers filenames. So in this example it would be:

```
/etc/sudoers.d/010-wheel
/etc/sudoers.d/100-wg-client
```

thereby ensuring that wg-client entries follow the wheel ones.

For convenience this is also noted in the sample file:

```
/etc/wg-client/sudoers.sample
```

```
chmod -440 /etc/sudoers.d/wg-client
```

The same thing asl can be achieved using doas or runo.

CHAPTER

THREE

RECENT CHANGES

8.0.3

- Typo in Readme

8.0.2

- Add missing dependency no PyCidr package used by the ssh listener

8.0.0

- Package / build management now uses meson/mesonpy
- New resolv-manager: - wirtten in C. - Runs as a background daemon managing /etc/resolv.conf - replaces the python inotify based monitor - see man page wg-client-resolv-manager
- You need to make a change. Please update PostUp / PostDown in wireguard config and use the helper scripts installed in /etc/wg-client: post-up.sh and post-down.sh. Create /etc/wg-client/wireguirs.resolv.conf with the DNS settings you want to use while VPN tunnel is up. Standard resolv.conf format (nameserver x.x.x.x)

7.2.0

- Drop unmaintained *netifaces* module - use our own code for what we need.

7.0.0

- Code Reorg
- Switch packaging from hatch to uv
- Add source checks for C-code as well as python
- Testing to confirm all working correctly on python 3.14.2
- C-code (wg-fix-resolve): Use more 'const' pointer to structs to improve security.
- Modern coding standards: PEP-8, PEP-257 and PEP-484 style and type annotations
- Use pyconcurrent module (additional dependency)

  Available on Github and AUR

CHAPTER

FOUR

CONFIGURATION

wg-client reads its configuration from

```
/etc/wg-client/config
```

Please copy the sample config and edit appropriately. The format is in *TOML* format.

```
# Sample
iface = 'wgc'
ssh_server = 'vpn.example.com'
ssh_pfx = '47'
```

This config file provides:

- iface - required

  Wireguard interface; defaults to *wgc*. It is *<iface>* of */etc/wireguard/<iface>.conf*

- ssh_server - optional

  Hostname of the remote ssh server accessible over the vpn; this is where the ssh listening port is run. Hostname must be accessible over the wg vpn.

- ssh_pfx - used with ssh_server

  1 or 2 digit number, 65 or smaller, to be used as ssh listening port number prefix. The port number is of the form PPxxx, with *PP* the prefix and *xxx* is taken from the last octet of the wireguard vpn internal IP address.

  The prefix can also be given as a range of numbers (*'n-m'*). In this case the prefix used is randomly chosen from that range

  Keep in mind that the largest port number is 65535, which limits *ssh_pfx* to be 65 or lower.

The port number chosen will be written to the log file.

The remote ssh host will then listen on *127.0.0.1:<port>*. It will also listen on *<remote-ip-address>:<port>* provided the remote ssh server permits it by having the sshd option set:

```
GatewayPorts yes
```

CHAPTER

FIVE

OPTIONS

5.1 wg-client

Summary of available options for wg-client.

Positional argument : Optional

- wireguard interface name

  Default interface is taken from *iface* in config file. The config file is chosen by first checking for *./etc/wg-client/config*[1] and then in */etc/wg-client/config*. If not found the wg interface defaults to *wgc*

Options:

These are all the options available in wg-client:

```
positional arguments:
  iface                 Optional wg interface (or test-dummy for test mode)

options:
  -h, --help            show this help message and exit
  --test                Test mode
  --wg-up               Turn wireguard on
  --wg-dn               Turn wireguard off
  --resolv-manager-start
                        Try (re)start resolv-manager daemon - development
↪option
  --ssh-start           Run ssh over vpn to create remote listener
  --ssh-stop            Kill ssh listener if its running
  --ssh-pfx SSH_PFX     ssh port(s). 2 digits: "nn" or "nn-mmm" for range
  --ssh-server SSH_SERVER
                        Remote ssh server to set up listening port
  --show-iface          Report the wg interface name
  --show-ssh-server     Report the ssh server name
  --show-ssh-running    Report if ssh is running
  --show-wg-running     Report if wireguard is running
  --show-resolv-manager
                        Report if resolv-manager is running
  --show-info           Display status - alias for --status
  --status              Display status
  --version             Display version
```

[1] Useful during development and testing

5.1.1 Uaing ssh

When asking wg-client to start ssh, it runs ssh to the remote host inside the vpn tunnel, sets it to listen on a remote port. Port number used is described in section *Configuration*.

Note that this blocks waiting for ssh. To stop ssh, simply make a separate invocation of *wg-client -ssh-stop*. If using the GUI tool, simply click the *Stop Ssh* button.

In the event that ssh connection is dropped, it will automatically be restarted. There are normal, quite common situations where ssh process can exit prematurely.

For example:

- After sleep/resume (longer than tcp timeout)
- if remote server sshd restarts (reboot for example)
- changing IP address (e.g. happens when location changes. e.g. Move from hotel to starbucks)

While attempting to reconnect ssh there is a waiting period between each attempt. Currently the wait time is 30 seconds.

The ssh port prefix can be 2 digits: “nn” or a range of 2 digit numbers “nn-mm”. When using a range, prefix used will be randomly drawn from the range. Maximum value is 65. This can also be set in the config file.

CHAPTER

SIX

LOG FILES

There are 3 kinds of log files: one for resolv-manager, one for command line app and one for the gui app.

resolv-manager logs are in /var/log/wg-client and while wg-client and wg-client-gui logs are in:

```
${HOME}/log/wg-client
${HOME}/log/wg-client-gui
```

All log files are rotated with the usual integer suffix scheme.

CHAPTER

SEVEN

APPENDIX

7.1 Installation

Available on:

- Github
- Archlinux AUR

On Arch you can build using the PKGBUILD provided in packaging directory or from the AUR package.

To build manually, clone the repo and do:

```
./scripts/do-build
./scripts/do-install [<destination>]
```

If destination is not specified it defaults to *build/pkg*

7.2 Dependencies

Run Time :

- python (3.13 or later)
- psutil (python-psutil)
- dateutil
- libcap
- pynotify (python-notify)
- openssl
- pyconcurrent
- PyQt6 / Qt6 (for gui)
- hicolor-icon-theme
- bash
- glibc

Building Package:

- git
- meson

- meson-python
- uv
- rsync

Optional for building docs: * sphinx * texlive-latexextra (archlinux packaguing of texlive tools)

7.3 Log files

Each application has it's own log file. These are located in users home directory :

```
${HOME}/log/wg-client
${HOME}/log/wg-client-gui
```

Each of the log files are rotated with companion log suffixed with *.1*

7.4 Philosophy

We follow the *live at head commit* philosophy as recommended by Google's Abseil team[1]. This means we recommend using the latest commit on git master branch.

7.5 License

Created by Gene C. and licensed under the terms of the GPL-2.0-or-later license.

- SPDX-License-Identifier: GPL-2.0-or-later
- SPDX-FileCopyrightText: © 2023-present Gene C <arch@sapience.com>

[1] https://abseil.io/about/philosophy#upgrade-support